

Data Modeling – Cumulative Dimensions, Struct & Array (Day 1 Lab)

Follow-Along Notes from DataExpert.io Free Boot Camp

Video: [YouTube Video – Data with Zach](#) Boot Camp: [DataExpert.io Community Academy](#)

1. Main Goal of the Lab

The purpose of this lab is to:

- Learn how to use:
 - `STRUCT`
 - `ARRAY`
- cumulative table design
- Reduce temporal duplication in tables
- Store historical records efficiently
- Make analytical queries easier and faster

The lab uses NBA player season data as the example.

2. Problem with Traditional Seasonal Tables

Initial table:

player_seasons

Typical structure:

player_name	season	pts	ast	reb
LeBron	2019	27	8	8
LeBron	2020	25	7	7
LeBron	2021	30	6	8

Problem

Player attributes repeat every season:

- height
- college
- birth country
- draft year
- player name

This creates:

- data duplication
- larger storage usage
- slower scans
- inefficient analytics

This is called:

Temporal Cardinality Explosion

As time increases → rows increase massively.

3. Key Concept – Separate Temporal vs Non-Temporal Data

Non-Temporal Attributes

These rarely change:

- player_name
- height
- college
- country
- draft info

Store once.

Temporal Attributes

These change every season:

- points

- assists
- rebounds
- games played
- season

Store historically.

4. Using STRUCT in PostgreSQL

The lab creates a custom STRUCT-like type.

Example:

```
CREATE TYPE season_stats AS (
  season INTEGER,
  gp INTEGER,
  pts REAL,
  reb REAL,
  ast REAL
);
```

Why?

Instead of many columns repeating for every row:

```
season | pts | reb | ast
```

we package them together.

Think of STRUCT as:

“A table inside a column.”

5. Using ARRAY

Instead of one row per season:

player	season
LeBron	2019

player	season
LeBron	2020
LeBron	2021

We create:

player	seasons
LeBron	[2019, 2020, 2021]

But each array item is actually a STRUCT.

Example:

```
[
  (2019, 82, 27.4, 8.5, 8.3),
  (2020, 67, 25.3, 7.8, 7.8)
]
```

6. Final Table Design

New table:

players

Example structure:

```
CREATE TABLE players (
  player_name TEXT,
  height TEXT,
  college TEXT,
  country TEXT,
  draft_year INTEGER,
  season_stats season_stats[],
  current_season INTEGER
);
```

Important Idea

One player = one row.

Historical seasons are stored inside:

```
season_stats[]
```

This is cumulative design.

7. What is Cumulative Table Design?

A cumulative table:

- keeps historical data together
- appends new information over time
- avoids duplicating static attributes

Instead of rebuilding history every day:

we continuously append seasonal information.

8. Incremental Loading Logic

Each new season:

1. Take yesterday's player row
2. Add today's season struct
3. Store updated array

Conceptually:

```
old_array + new_season_record
```

9. Full Outer Join Pattern

The lab uses:

FULL OUTER JOIN

Why?

Need to handle:

- existing active players
- retired players
- rookies/new players

Cases:

Case	Action
Existing player + new season	append array
Existing player no new season	keep old data
New player	create fresh array

10. ARRAY Concatenation

Important SQL pattern:

```
old_array || new_element
```

Example:

```
ARRAY[ROW(2023, 80, 25.0, 7.0, 6.0)::season_stats]
```

This appends a season record.

11. COALESCE Usage

The lab heavily uses:

```
COALESCE()
```

Purpose:

- handle NULLs
- merge old/new player data
- preserve historical records

Example:

```
COALESCE(today.player_name, yesterday.player_name)
```

12. Handling Retired Players

Important concept.

Retired players:

- still exist historically
- should not get fake NULL seasons appended

Good cumulative design:

✓ Preserve history ✗ Avoid unnecessary empty records

13. Benefits of This Design

Storage Efficiency

Avoids repeated attributes.

Faster Analytics

Historical data is grouped together.

Better Compression

Works well with:

- Parquet
- run-length encoding
- columnar storage

Easier Historical Queries

No massive GROUP BY operations.

14. Tradeoff – Compactness vs Usability

A major theme in the boot camp.

Compact	Usable
arrays	flat tables
structs	simple columns
compressed	analyst-friendly

Nested data is:

✓ efficient ✗ harder to query

Flat data is:

✓ easy to use ✗ expensive to store

15. Querying Arrays

Important PostgreSQL function:

```
UNNEST()
```

Used to explode arrays back into rows.

Example:

```
SELECT player_name, UNNEST(season_stats)
FROM players;
```

16. Analytical Possibilities

With cumulative design you can:

- track player improvement
- compare season-over-season performance
- analyze trends
- calculate rolling metrics
- build timelines easily

Without expensive joins/grouping.

17. Real Industry Relevance

These patterns are used in:

- Airbnb
- Uber
- Netflix
- modern data warehouses
- lakehouse systems

Especially useful for:

- slowly changing historical data
 - event histories
 - user activity timelines
 - machine learning feature stores
-

18. Important SQL Concepts Used in the Lab

CREATE TYPE

```
CREATE TYPE
```

Creates custom STRUCT type.

ARRAY

```
ARRAY[]
```

Creates arrays.

ROW

```
ROW(...)
```

Creates struct records.

CASTING

```
::season_stats
```

Casts row into custom type.

FULL OUTER JOIN

Combines old and new records.

COALESCE

Handles NULL logic.

UNNEST

Explodes arrays into rows.

19. Mental Model to Remember

Traditional Design

One row = one season

Cumulative Design

One row = one entity
Array = history

This is the biggest takeaway from the lab.

20. Simple End-to-End Flow

Step 1

Raw table:

player_seasons

Step 2

Create STRUCT type:

season_stats

Step 3

Group seasons into ARRAY.

Step 4

Build cumulative players table.

Step 5

Append new seasons incrementally.

21. Interview-Friendly Explanations

What is a cumulative table?

A table that stores the historical evolution of an entity in a compact format, usually by appending historical records into arrays or nested structures.

Why use STRUCT and ARRAY?

To reduce duplication, improve compression, and efficiently store historical data.

Why not keep flat rows?

Flat rows are easier to query but become extremely expensive and repetitive at scale.

When should you use cumulative design?

Use it when:

- historical tracking matters
 - entities evolve over time
 - data size becomes large
 - analytics frequently need history
-

22. Key Takeaways

Core Learning

- STRUCT = grouped attributes
 - ARRAY = historical collection
 - cumulative tables = compact historical storage
-

Most Important Idea

Instead of:

duplicating rows over time

Store:

one entity + historical array

SQL Skills Learned

- custom types
 - arrays
 - row constructors
 - joins
 - incremental loading
 - historical modeling
-

23. Recommended Practice

Try rebuilding the same design for:

- customer orders
- employee history
- subscription plans
- IoT sensor history
- website sessions

This helps deeply understand cumulative modeling.

24. Useful Resources

- [DataExpert.io Academy](#)
 - [Data with Zach YouTube Channel](#)
 - [PostgreSQL Arrays Documentation](#)
 - [PostgreSQL Composite Types Documentation](#)
-

25. Final One-Line Summary

Cumulative dimensional modeling uses STRUCT + ARRAY to store historical data compactly by keeping one entity per row and placing time-based history inside nested collections.